Meshtastic FLRC & GFSK Fast Modems for SX1280

Oct 24, 2026

Summary

A Meshtastic 2.7.26 fork adds eight FLRC and GFSK presets as modes independent of LoRa; in two-node tests they cut acknowledged-message latency to ~160 ms, against 198 ms for the fastest LoRa preset and 587 ms for MEDIUM_SLOW.

- **What changed:** 8 new presets (4 FLRC, 4 GFSK), an SX1280 driver path for them, FLRC fragmentation for packets over 127 bytes, faster encryption on the nRF52, DFU without pressing reset, and a new board variant. LoRa code paths are unchanged.
- **Speed:** every fast preset delivers ~24 kb/s of streamed broadcasts with 19–20 of 20 packets received. The radio could carry 260–2000 kb/s; the Meshtastic stack (~50–60 ms per packet) is now the limit.
- **Status:** both boards run the fork. Open issue: FLRC loses 1–2 of every 20 ACKs, which retransmission recovers after about 1 s.

Hardware and baseline

Two Faketec 2.4G boards were tested side by side in one room at −18 dBm, receiving at −29 to −32 dBm RSSI.

Item	Value
Board	ProMicro nRF52840 (Faketec 2.4G), Adafruit UF2 bootloader
Radio module	Ebyte E28-2G4M27S (Semtech SX1280 + PA/LNA, RXEN/TXEN switched)
SX1280 pins	SPI MISO = P0.02, MOSI = P1.15, SCK = P1.11, CS = P1.13, DIO1 = P0.10, BUSY = P0.29, RESET = P0.09, RXEN = P0.17, TXEN = P1.06
Base firmware	Meshtastic 2.7.26 (commit 54e0d8d), PlatformIO env `nrf52_promicro_diy_sx1280`
Region / frequency	LORA_24, override frequency 2482.0 MHz

Item	Value
TX power in tests	−18 dBm on both nodes (originally 1 dBm on COM14 and 10 dBm on COM27)
Nodes	COM14 = sender (A), COM27 = receiver (B)

At −11 dBm RSSI the receiver LNA saturates, so power was dropped to −18 dBm for every same-room test.

Firmware changes

The patch touches 18 files (+647/−21 lines); it adds the fast modes beside LoRa and leaves every LoRa preset on its original code path.

New presets

The presets extend the `ModemPreset` enum with local values; the official apps and protobufs do not know them, so they are set by number.

Preset	No.	Modulation	Bitrate (kb/s)	Bandwidth (kHz)	Coding rate	Sensitivity (dBm)
FLRC_TURBO	64	FLRC	1300	1200	1/1	−96
FLRC_FAST	65	FLRC	1300	1200	1/2	−99
FLRC_MEDIUM	66	FLRC	650	600	1/2	−104
FLRC_LONG	67	FLRC	260	300	1/2	−106
GFSK_TURBO	72	GFSK	2000	2400	–	−83
GFSK_FAST	73	GFSK	1000	2400	–	−87
GFSK_MEDIUM	74	GFSK	500	1200	–	−90
GFSK_LONG	75	GFSK	250	600	–	−92

FLRC CR 3/4 was left out because it lost 20–30% of exchanges when TX and RX alternate. A fast preset needs a wide region (LORA_24) and an SX128x radio; otherwise the node falls back to LONG_FAST.

Radio configuration

- **Frame:** 24-bit preamble, 4-byte sync word `2D 4B D4 93` (kept to reject 2.4 GHz noise), 2-byte CRC, variable length; GFSK uses modulation index 0.5 (deviation = bitrate/4), FLRC uses BT 0.5 shaping.

- **FLRC fragmentation:** FLRC frames carry at most 127 bytes, so every FLRC packet gets a 1-byte header. `0x00` marks a whole packet; otherwise the byte holds the fragment index, the fragment count and a 3-bit tag, and the payload is split into near-equal parts. The receiver reassembles them and drops incomplete sets.
- **Channel access:** a 1 ms CSMA slot instead of LoRa's CAD-based slot. The channel counts as busy while a sync word is being received or RSSI is 10 dB above sensitivity or the noise floor.
- **Timing:** 30 ms between FLRC fragments; minimum gap before a new transmission of 30 ms for GFSK and 10 ms for FLRC; 1000 ms retransmission base instead of 4500 ms.
- **Link metrics:** RSSI comes from the packet; SNR is estimated as RSSI − sensitivity − 20 dB, clamped to −20…+10 dB, so SNR-weighted rebroadcast delays still work.

Files

File	Change
`src/mesh/generated/meshtastic/config.pb.h`	Enum values 64–67 and 72–75
`src/DisplayFormatters.cpp`	Names for the new presets (FlrcFast / FlrcF…), which also become channel names
`src/mesh/FastModem.h` (new)	`RadioModulation` enum and `FastModemParams` struct
`src/mesh/MeshRadio.h`	Preset table: modulation, bitrate, bandwidth, coding rate, sensitivity
`src/mesh/RadioInterface.h/.cpp`	Fast-mode selection with fallback, slot time, TX gaps, retransmission time
`src/mesh/RadioLibInterface.h/.cpp`	Hooks for multi-frame transmit and partial receive (`NEXT_FRAME`)
`src/mesh/SX128xInterface.h/.cpp`	FLRC/GFSK setup, fragmentation, CSMA, RSSI/SNR, airtime
`src/mesh/CryptoEngine.h/.cpp`	Cache of 4 shared keys: direct-message decryption 81 ms → 11 ms
`src/platform/nrf52/NRF52CryptoEngine.cpp`	Keeps the CC310 crypto block powered between calls: AES 98 ms → ~10 ms

File	Change
`src/platform/nrf52/main-nrf52.cpp`	DFU entry that this bootloader honours (no reset button needed)
`src/RedirectablePrint.cpp`	Millisecond timestamps in logs (debugging only)
`variants/nrf52840/diy/nrf52_promicro_diy_sx1280/` (new)	Board variant for the ProMicro + E28 SX1280 module

Test method

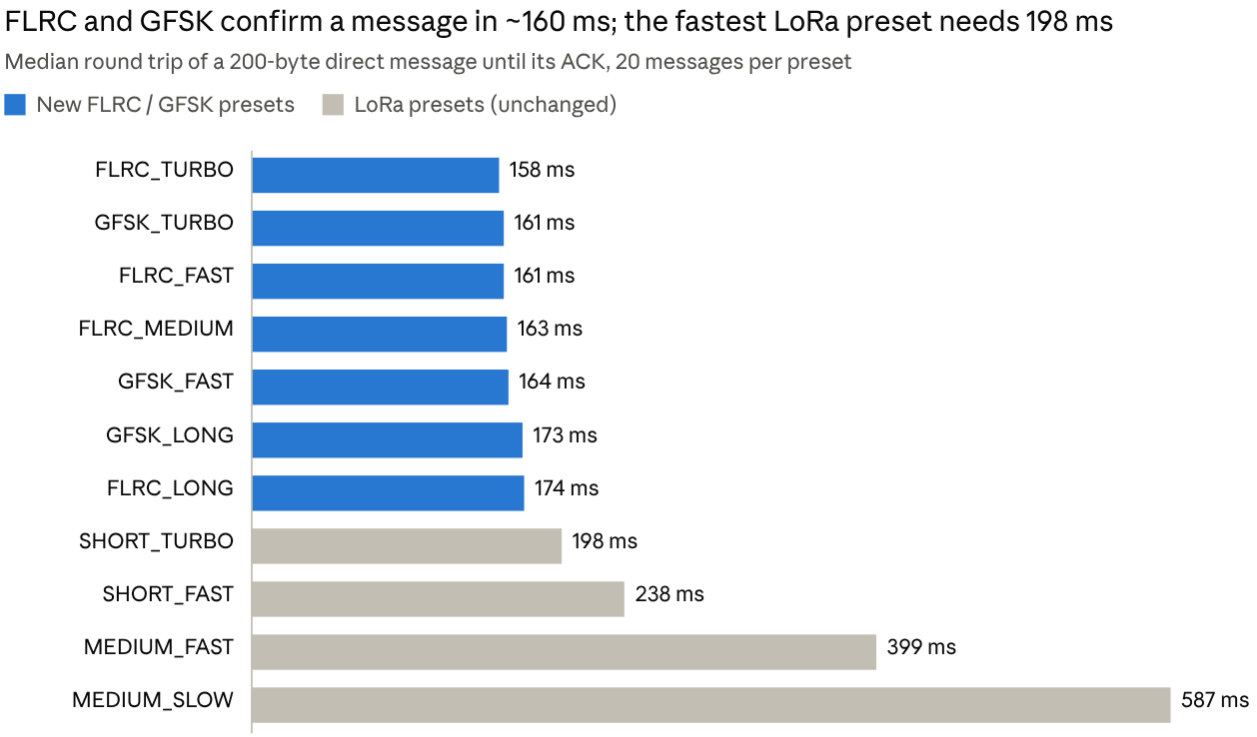
Each preset was measured end to end through the Meshtastic serial API with `mesh_speed.py`, 20 messages of 200 bytes per test.

1. Both nodes are switched to the preset (config write and reboot), then the script waits 45 s for boot to finish.
2. The nodes exchange NodeInfo so each has the other's public key, and 3 warm-up messages are sent.
3. **ACK test:** A sends 20 direct messages to B, each waiting for B's acknowledgement before the next; the script logs success and round-trip latency.
4. **Stream test:** A queues 20 broadcasts without ACK back to back; the script counts how many reach B's PC and the rate from first send to last arrival.

Median ACK latency is the most stable figure. Messages per second in the ACK test also include Python and USB overhead and swing between runs, so it is not reported. `mesh_trace.py` captured B's raw log with millisecond timestamps to locate losses.

Results

Every new preset beats every LoRa preset on latency, and all of them stream 21–26 kb/s with at most 1 of 20 broadcasts lost.



mesh_speed.py runs on 2026-10-24 · COM14 → COM27, −18 dBm, final firmware timing

Preset	ACK delivered	Stream received	Stream rate (kb/s)
GFSK_LONG	20/20	20/20	25.6
FLRC_FAST	19/20	20/20	24.7
GFSK_TURBO	19/20	19/20	24.5
GFSK_FAST	20/20	19/20	24.4
FLRC_TURBO	18/20	19/20	23.6
FLRC_MEDIUM	18/20	20/20	23.5
FLRC_LONG	19/20	20/20	20.8
SHORT_TURBO (LoRa)	20/20	16/20	18.8
SHORT_FAST (LoRa)	20/20	17/20	13.8
MEDIUM_FAST (LoRa)	20/20	20/20	7.2
MEDIUM_SLOW (LoRa)	20/20	20/20	3.9

GFSK_MEDIUM was only measured before the 30 ms GFSK gap (135 ms ACK latency, 11/20 streamed), so it is left out. SHORT_TURBO streamed 19/20 at 22.2 kb/s in an earlier run of the same LoRa code.

Tuning steps

Step	Effect
Shared-key cache + CC310 kept on	Direct-message decryption 81 → 11 ms, AES 98 → ~10 ms
FLRC fragment gap 3 → 30 ms	FLRC stream 29/30 instead of dropped second fragments
Wait 45 s after reboot (GPS probing)	GFSK stream 6–7/20 → 11–12/20; GFSK ACK median 237–370 → 129–137 ms
GFSK minimum TX gap 10 → 30 ms	GFSK stream 11–12/20 → 19–20/20 (~25 kb/s); ACK median +30 ms
FLRC gap kept at 10 ms	A 30 ms gap cut FLRC stream to ~19 kb/s, so it stays at 10 ms

Bottlenecks and known issues

The radio is no longer the limit: a 240-byte packet spends 2–8 ms on air, but Meshtastic needs ~50–60 ms per packet, so every fast preset tops out near 24 kb/s.

Bottleneck	Effect	Possible fix
Meshtastic stack per packet (router, crypto, phone queue, serial API)	~24 kb/s ceiling; the same radios reached 934 kb/s (FLRC) and 1344 kb/s (GFSK) PC to PC with custom firmware	Profile the router path; batch packets to the phone API
Receive is serviced from the cooperative main loop	A frame not read within ~30 ms is overwritten by the next, which forces the 30 ms gaps	Read the SX1280 buffer from a high-priority task right after the RX interrupt
GPS probing for ~35 s after boot	Main loop blocked; packets lost right after every reboot or preset change	Disable GPS (`position.gps_mode NOT_PRESENT`) on boards without one
FLRC ACK losses	1–2 of 20 direct messages lose their ACK; recovered by retransmission after ~1 s	Not yet diagnosed; capture both nodes' logs during a failure

Bottleneck	Effect	Possible fix
FLRC 127-byte frame limit	Packets over 126 bytes go in 2 fragments with a 30 ms gap	Inherent to FLRC; GFSK carries 255 bytes in one frame
Lower GFSK sensitivity	GFSK_TURBO (−83 dBm) has 23 dB less link budget than FLRC_LONG (−106 dBm)	Choose the preset by range, not only speed

Range was not tested: both boards sat in one room at −18 dBm, so the sensitivity figures come from the datasheet.

Instructions

All nodes in a test must run this firmware, use region LORA_24 and share the same preset.

Build

1. Get the Meshtastic firmware 2.7.26 source (commit 54e0d8d, the `v2.7.26.54e0d8d` release) and apply the patch from its root folder:

```
git apply meshtastic_fast_modems.patch
```

2. Build the new variant with PlatformIO:

```
pio run -e nrf52_promicro_diy_sx1280
```

On Windows, long dependency paths can break the build; map the source folder to a drive letter first (`subst W: <folder>`). The UF2 lands in `.pio/build/nrf52_promicro_diy_sx1280/`.

Flash

1. Put the board in bootloader mode without touching it (a board already running this fork or stock Meshtastic accepts the command):

```
meshtastic --port COM14 --enter-dfu
```

2. Copy the `.uf2` file to the USB drive that appears; the board reboots by itself. Double-tapping reset also enters the bootloader. `flash.py --meshtastic --uf2 <file>` does both steps for every connected board.

Change modulation and speed

Set the preset by number, since the official CLI and apps do not know the new names; the node reboots on save.

```
meshtastic --port COM14 --set lora.modem_preset 66
```

Family	Presets and numbers
FLRC	FLRC_TURBO 64, FLRC_FAST 65, FLRC_MEDIUM 66, FLRC_LONG 67
GFSK	GFSK_TURBO 72, GFSK_FAST 73, GFSK_MEDIUM 74, GFSK_LONG 75
LoRa	LONG_FAST 0, MEDIUM_SLOW 3, MEDIUM_FAST 4, SHORT_SLOW 5, SHORT_FAST 6, SHORT_TURBO 8

- **Region:** `--set lora.region LORA_24`; any other region makes a fast preset fall back to LONG_FAST.
- **Channel:** each preset has its own default channel name, so nodes on different presets never hear each other.
- **Apps:** the Android and iOS apps show these presets as unknown; editing LoRa settings there overwrites the preset.
- **Power:** −18 dBm suits one room. For range tests raise it with `--set lora.tx_power 3`, the highest chip setting the firmware allows (the E28 module's PA amplifies it).
- **After a reboot:** wait ~45 s, or disable GPS with `--set position.gps_mode NOT_PRESENT`, before measuring.

Run the speed test

```
python mesh_speed.py --a COM14 --b COM27 --presets
FLRC_FAST,GFSK_FAST,SHORT_TURBO --n 20 --size 200
```

Results print per preset and are saved to `mesh_results.json`. `mesh_trace.py --broadcast --n 15` captures the receiver's raw log while the sender transmits.

Restore the original setup

Return the nodes to MEDIUM_SLOW and their original power (1 dBm on COM14, 10 dBm on COM27), or load the saved configs with `meshtastic --port COM14 --configure meshtastic_backup/COM14_config.yaml`. The backup files hold private keys, so keep

them off shared drives. Stock firmware can be reflashed the same way as the fork.

Files

File	Contents
`meshtastic_fast_modems.patch`	All firmware changes against Meshtastic 2.7.26 (applies cleanly with `git apply`)
`meshtastic-2.7.26-sx1280-fastmodems.uf2`	Ready-to-flash firmware for the Faketec 2.4G board
`mesh_speed.py`	Preset switching, ACK test and stream test; writes JSON results
`mesh_trace.py`	Receiver log capture with millisecond timestamps
`flash.py`	Hands-off flashing of every connected board via the DFU command
`mesh_results_v2.json` … `v4.json`	Raw results of the runs in this report
`meshtastic_backup/COM14_config.yaml`, `COM27_config.yaml`	Original node configs (contain private keys; keep local)

The earlier PC-to-PC modem tests are in Informe SX1280 FLRC / LoRa / GFSK.